

Practical: 5	
26/08/2026	Implementation of Knapsack problem using dynamic programming

Aim: Implementation of Knapsack problem using dynamic programming

Algorithm:

Algorithm:
Input : n (number of items), w (capacity), $w[i]$ (weights),
 $v[i]$ (values)
Output : Maximum value and selected items
Step-1: Set $dp[i][0] = 0$ and $dp[0][i] = 0$
Step-2: For $i \leftarrow 1$ to n and $j \leftarrow 1$ to w
Step-3: If $w[i] \leq j$
 $dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$
Step-4: Else, $dp[i][j] = dp[i-1][j]$
Step-5: Return $dp[n][w]$ as maximum value

Program:

```
#include<iostream>
using namespace std;
```

```
class Knapsack {

private:
    int w[10];    // weights
    int v[10];    // values
    int n;        // number of items
    int W;        // capacity
    int dp[10][100]; // dp table

public:

    // — Get Input —
    void getInput() {
        cout << "Enter number of
items : ";
        cin >> n;
        cout << "Enter knapsack
capacity : ";
        cin >> W;
        for(int i = 1; i <= n; i++) {
            cout << "Item " << i << "
weight and value : ";
            cin >> w[i] >> v[i];
        }
    }

    // — Build dp table —
    void solve() {

        // base case — row 0 and col 0
        for(int i = 0; i <= n; i++)
            dp[i][0] = 0;
        for(int j = 0; j <= W; j++)
            dp[0][j] = 0;
```



```
// fill table
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= W; j++) {

        if(w[i] <= j) {
            // take or not take
            dp[i][j] = max(
                v[i] + dp[i-1][j-
w[i]],
                dp[i-1][j]
            );
        } else {
            // cannot take item i
            dp[i][j] = dp[i-1][j];
        }
    }
}

// — Print dp table —
void printTable() {
    cout << "\nDP Table:" <<
endl;
    cout << " ";
    for(int j = 0; j <= W; j++)
        cout << j << " ";
    cout << endl;

    for(int i = 0; i <= n; i++) {
        cout << "i=" << i << " ";
        for(int j = 0; j <= W; j++)
            cout << dp[i][j] << " ";
        cout << endl;
    }
}
```

```
// — Find selected items —
void findItems() {
    cout << "\nSelected Items:"
<< endl;
    int j = W;
    for(int i = n; i >= 1; i--) {
        if(dp[i][j] != dp[i-1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN " <<
endl;
            j = j - w[i];
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " <<
endl;
        }
    }
}

void showResult() {

    cout << "\nMaximum Value =
    << dp[n][W] << endl;
}

};

int main() {

    Knapsack k;
```

```
k.getInput();  
k.solve();  
k.printTable();  
k.showResult();  
k.findItems();  
  
return 0;  
}
```

Output:

```
Enter number of items : 4  
Enter knapsack capacity : 3  
Item 1 weight and value : 7  
6  
Item 2 weight and value : 8  
3  
Item 3 weight and value : 5  
1  
Item 4 weight and value : 3  
7  
  
DP Table:  
    0  1  2  3  
i=0  0  0  0  0  
i=1  0  0  0  0  
i=2  0  0  0  0  
i=3  0  0  0  0  
i=4  0  0  0  7  
  
Maximum Value = 7  
  
Selected Items:  
Item 4 (w=3, v=7) TAKEN  
Item 3 (w=5, v=1) NOT taken  
Item 2 (w=8, v=3) NOT taken  
Item 1 (w=7, v=6) NOT taken
```

Time Complexity:

Best case : The DP table is filled for all n items and W capacities.

Time Complexity:

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (w[i] <= j) {
            dp[i][j] = max (
                v[i] + dp[i-1][j-w[i]]
            );
            dp[i][j] = max (v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]);
        }
        else
            dp[i][j] = dp[i-1][j];
    }
}

T(n, w) = n * w * O(1)
T(n, w) = O(nw)
    
```

Average case : On average, all $n \times W$ entries of the DP table are calculated.

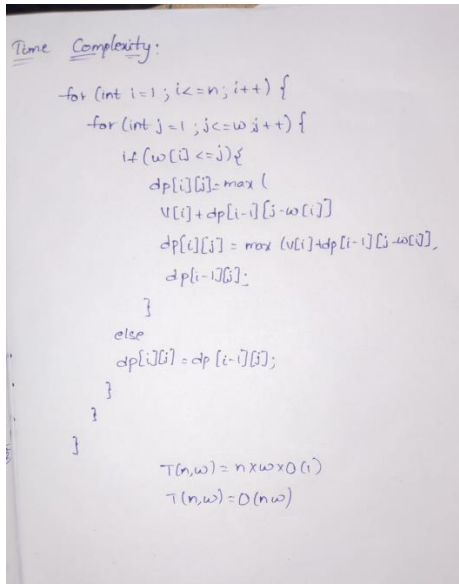
Time Complexity:

```

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (w[i] <= j) {
            dp[i][j] = max (
                v[i] + dp[i-1][j-w[i]]
            );
            dp[i][j] = max (v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]);
        }
        else
            dp[i][j] = dp[i-1][j];
    }
}

T(n, w) = n * w * O(1)
T(n, w) = O(nw)
    
```

Worst case : In the worst case also, all $n \times W$ entries of the DP table are calculated.



```
Time Complexity:
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {
        if (w[i] <= j) {
            dp[i][j] = max (
                v[i] + dp[i-1][j-w[i]],
                dp[i-1][j]
            );
        }
        else
            dp[i][j] = dp[i-1][j];
    }
}

T(n,w) = n * w * O(1)
T(n,w) = O(nw)
```

Conclusion:

The 0/1 Knapsack problem was successfully implemented using the Dynamic Programming algorithm. The algorithm finds the maximum profit by considering whether to include or exclude each item.